MA 551 - Homework 3

Adam Field

March 31, 2026

1 Problem 1: Cauchy Distribution and Rejection Sampling

1.1 Inversion Method for the Cauchy Distribution

The Cauchy(α) distribution has CDF

$$F(x) = \frac{1}{2} + \frac{1}{\pi}\arctan\left(\frac{x}{\alpha}\right).$$

Setting $U = F(X)$ and solving for X gives the inversion formula

$$X = \alpha \tan\!\big(\pi(U - \tfrac{1}{2})\big), \qquad U \sim \text{Uniform}(0,1).$$

```r
rcauchy_inv <- function(n, alpha = 1) {
  u <- runif(n)
  alpha * tan(pi * (u - 0.5))
}

# Verify against built-in Cauchy quantiles
set.seed(42)
x_test   <- rcauchy_inv(10000, alpha = 1)
ks_pval  <- ks.test(x_test, "pcauchy", location = 0, scale = 1)$p.value
cat(sprintf("KS test p-value vs Cauchy(0,1): %.4f\n", ks_pval))
```

```
## KS test p-value vs Cauchy(0,1): 0.2077
```

The large p-value from the Kolmogorov-Smirnov test confirms that the inversion sample is consistent with the Cauchy$(0,1)$ distribution.

1.2 Optimal Cauchy Envelope for N(0,1) Rejection Sampling

To generate $X \sim N(0,1)$ using a Cauchy(α) envelope we require $\phi(x) \le k \cdot h_\alpha(x)$ for all x, where $\phi(x) = (2\pi)^{-1/2}e^{-x^2/2}$ is the standard normal density and $h_\alpha(x) = \alpha/[\pi(\alpha^2 + x^2)]$ is the Cauchy(α) density. The minimum valid scaling constant is

$$k(\alpha) = \sup_x \frac{\phi(x)}{h_\alpha(x)} = \sup_x \frac{\pi(\alpha^2 + x^2)\, e^{-x^2/2}}{\alpha\sqrt{2\pi}}.$$

Differentiating the numerator with respect to x and setting it to zero yields $x\, e^{-x^2/2}(2 - \alpha^2 - x^2) = 0$, so the interior critical point satisfies $x^{*2} = 2 - \alpha^2$, which is real for $\alpha < \sqrt{2}$. Substituting $x^{*2} = 2 - \alpha^2$ back gives

$$k(\alpha) = \frac{\sqrt{2\pi}\, e^{\alpha^2/2-1}}{\alpha}.$$

To find the α that minimises $k(\alpha)$ we differentiate:

$$\frac{\mathrm{d}}{\mathrm{d}\alpha}\left[\frac{e^{\alpha^2/2-1}}{\alpha}\right] = \frac{e^{\alpha^2/2-1}(\alpha^2 - 1)}{\alpha^2} = 0 \implies \alpha^* = 1.$$

At $\alpha^* = 1$ the optimal constant is

$$k^* = \sqrt{2\pi}\, e^{-1/2} = \sqrt{\frac{2\pi}{e}} \approx 1.5203,$$

giving a theoretical acceptance probability of $1/k^* = \sqrt{e/(2\pi)} \approx 65.8\%$.

```r
alpha_opt <- 1
k_opt     <- sqrt(2 * pi / exp(1))
cat(sprintf("Optimal alpha: %.4f\n", alpha_opt))
```

```
## Optimal alpha: 1.0000
```

```r
cat(sprintf("Optimal k:     %.6f\n", k_opt))
```

```
## Optimal k:     1.520347
```

```r
cat(sprintf("Theoretical acceptance probability: %.4f\n", 1 / k_opt))
```

```
## Theoretical acceptance probability: 0.6577
```

```r
rnorm_cauchy_rej <- function(N, alpha, k) {
  samples    <- numeric(N)
  n_accepted <- 0
  n_rejected <- 0

  while (n_accepted < N) {
    x_prop <- rcauchy_inv(1, alpha)
    u      <- runif(1)
    f_x    <- dnorm(x_prop)
    h_x    <- alpha / (pi * (alpha^2 + x_prop^2))
    if (u <= f_x / (k * h_x)) {
      n_accepted <- n_accepted + 1
      samples[n_accepted] <- x_prop
    } else {
      n_rejected <- n_rejected + 1
    }
  }
  list(samples = samples, n_rejected = n_rejected)
}

set.seed(42)
res_p1 <- rnorm_cauchy_rej(1000, alpha_opt, k_opt)

cat(sprintf("Total rejections: %d\n", res_p1$n_rejected))
```

```
## Total rejections: 516
```

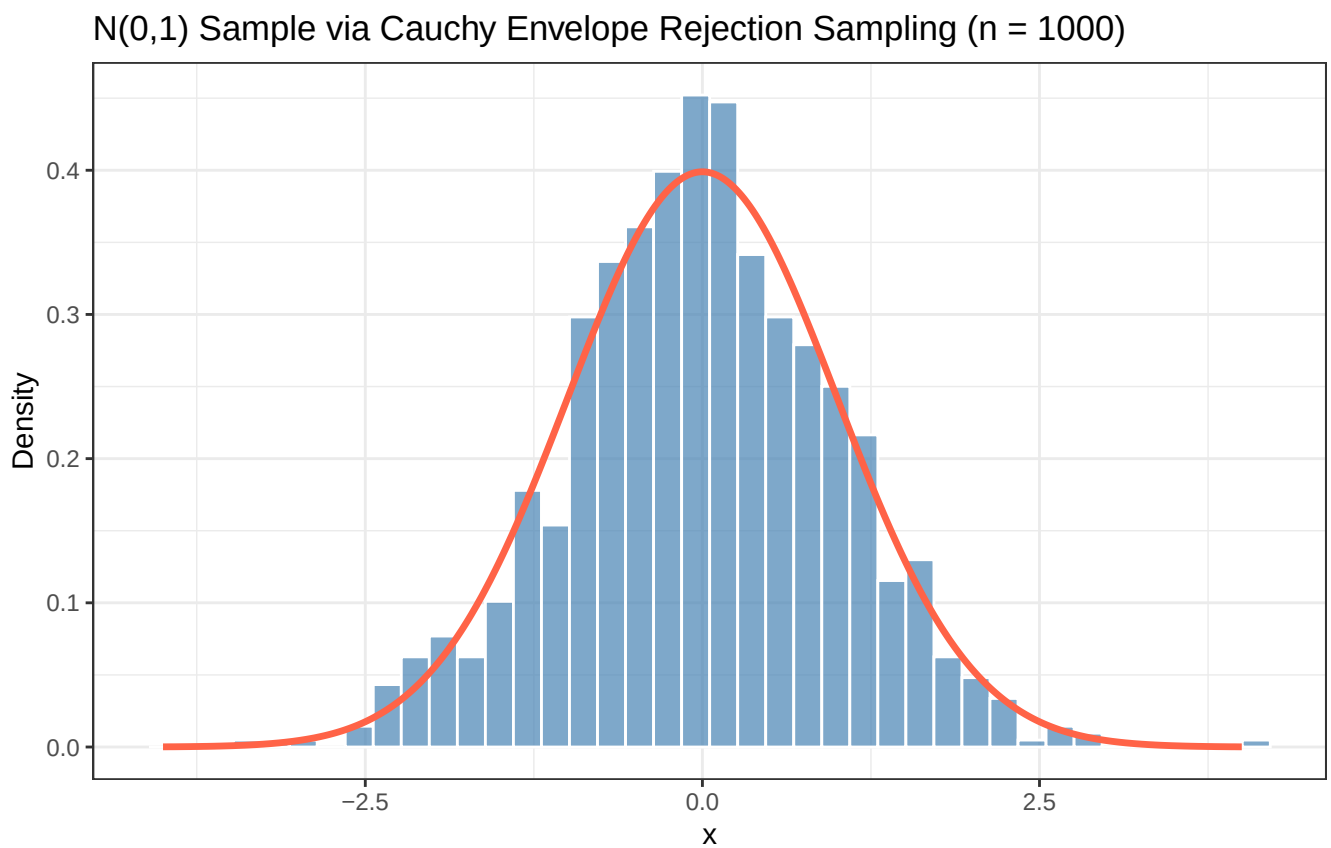
```r
cat(sprintf("Empirical rejection rate: %.4f\n",
            res_p1$n_rejected / (1000 + res_p1$n_rejected)))
```

```
## Empirical rejection rate: 0.3404
```

```r
x_grid   <- seq(-4, 4, length.out = 400)
df_theo  <- data.frame(x = x_grid, y = dnorm(x_grid))
df_samp  <- data.frame(x = res_p1$samples)

ggplot(df_samp, aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins = 40, fill = "steelblue", alpha = 0.7, color = "white") +
  geom_line(data = df_theo, aes(x = x, y = y),
            color = "tomato", linewidth = 1.2) +
  labs(title = "N(0,1) Sample via Cauchy Envelope Rejection Sampling (n = 1000)",
       x = "x", y = "Density") +
```

```r
  theme_bw()
```



The empirical rejection rate is close to the theoretical value of approximately 34.2%. The histogram follows the superimposed $N(0,1)$ density closely, confirming that the algorithm correctly produces standard normal samples.

2 Problem 2: Conditional Gamma Distribution

2.1 Part (a): Density of Gamma(2,1) Conditional on X > 4

The Gamma$(2,1)$ distribution has density $f_{\text{Gamma}}(x) = x\,e^{-x}$ for $x > 0$. By integration by parts,

$$P(X > 4) = \int_4^\infty x\,e^{-x}\,\mathrm{d}x = \Big[-x\,e^{-x}\Big]_4^\infty + \int_4^\infty e^{-x}\,\mathrm{d}x = 4e^{-4} + e^{-4} = 5e^{-4}.$$

The conditional density is therefore

$$f(x) = \frac{f_{\text{Gamma}}(x)}{P(X > 4)} = \frac{x\,e^{-x}}{5\,e^{-4}} = \frac{x}{5}\,\exp\big\{-(x-4)\big\}, \quad x > 4. \qquad \blacksquare$$

One may verify that f integrates to one: setting $u = x - 4$ gives $\frac{1}{5}\int_0^\infty (u+4)e^{-u}\,\mathrm{d}u = \frac{1}{5}[\Gamma(2) + 4] = 1$.

2.2 Part (b): Envelope Densities and Optimal Constants

2.2.1 Envelope $h_1(x)$: Exponential(1/2) Conditional on X > 4

For $Y \sim \text{Exp}(1/2)$ with density $\frac{1}{2}e^{-y/2}$,

$$P(Y > 4) = \int_4^\infty \tfrac{1}{2}e^{-y/2}\,\mathrm{d}y = e^{-2}.$$

The conditional density is

$$h_1(x) = \frac{\frac{1}{2}e^{-x/2}}{e^{-2}} = \frac{1}{2}\,e^{-(x-4)/2}, \quad x > 4.$$

The ratio to be maximised is

$$\frac{f(x)}{h_1(x)} = \frac{(x/5)\,e^{-(x-4)}}{(1/2)\,e^{-(x-4)/2}} = \frac{2x}{5}\,e^{-(x-4)/2}.$$

Its derivative equals $(2/5)\,e^{-(x-4)/2}(1 - x/2)$, which is strictly negative for $x > 4$. Hence f/h_1 is strictly decreasing and its maximum is attained at $x = 4$:

$$k_1 = \frac{2 \cdot 4}{5} = \frac{8}{5} = 1.6.$$

2.2.2 Envelope $h_2(x)$: Standard Cauchy Conditional on X > 4

For the standard Cauchy density $1/[\pi(1 + x^2)]$,

$$P(X_C > 4) = \int_4^\infty \frac{\mathrm{d}x}{\pi(1+x^2)} = \frac{1}{\pi}\Big[\arctan x\Big]_4^\infty = \frac{1}{\pi}\Big(\frac{\pi}{2} - \arctan 4\Big).$$

Writing $C_2 = \pi/2 - \arctan(4)$, the conditional density is

$$h_2(x) = \frac{1/(\pi(1+x^2))}{C_2/\pi} = \frac{1}{C_2(1+x^2)}, \quad x > 4.$$

The ratio to be maximised is

$$\frac{f(x)}{h_2(x)} = \frac{(x/5)\,e^{-(x-4)}}{1/[C_2(1+x^2)]} = \frac{C_2}{5}\,x(1+x^2)\,e^{-(x-4)}.$$

Differentiating $x(1+x^2)e^{-(x-4)}$ with respect to x yields $e^{-(x-4)}\big(1 - x + 3x^2 - x^3\big)$. At $x = 4$ this equals $e^0(1 - 4 + 48 - 64) = -19 < 0$, and the cubic $x^3 - 3x^2 + x - 1$ is strictly positive for $x > 4$ (so the bracketed factor is strictly negative there). Therefore f/h_2 is decreasing on $(4, \infty)$ and its maximum is at $x = 4$:

$$k_2 = \frac{C_2}{5} \cdot 4 \cdot 17 \cdot 1 = \frac{68\,C_2}{5}.$$

```r
C2 <- pi/2 - atan(4)
k1 <- 8/5
k2 <- 68 * C2 / 5

cat(sprintf("C2 = pi/2 - arctan(4) = %.6f\n", C2))
```

```
## C2 = pi/2 - arctan(4) = 0.244979
```

```r
cat(sprintf("k1 = %.4f  (acceptance rate = %.4f)\n", k1, 1/k1))
```

```
## k1 = 1.6000  (acceptance rate = 0.6250)
```

```r
cat(sprintf("k2 = %.4f  (acceptance rate = %.4f)\n", k2, 1/k2))
```

```
## k2 = 3.3317  (acceptance rate = 0.3001)
```

2.3 Part (c): rej_sample Implementation

The inverse CDF of h_1 follows from $H_1(x) = 1 - e^{-(x-4)/2}$:

$$H_1^{-1}(u) = 4 - 2\log(1-u).$$

For h_2, setting $H_2(x) = [\arctan(x) - \arctan(4)]/C_2 = u$ gives

$$H_2^{-1}(u) = \tan\big(\arctan(4) + u\,C_2\big).$$

```r
# Target density f(x)
fx <- function(x) ifelse(x > 4, (x / 5) * exp(-(x - 4)), 0)

# Envelope h1(x): Exp(1/2) shifted to start at 4
h1x <- function(x) ifelse(x > 4, 0.5 * exp(-0.5 * (x - 4)), 0)

# Inverse CDF of h1
ihx1 <- function(u) 4 - 2 * log(1 - u)

# Envelope h2(x): standard Cauchy conditional on X > 4
h2x <- function(x) ifelse(x > 4, 1 / (C2 * (1 + x^2)), 0)

# Inverse CDF of h2
ihx2 <- function(u) tan(atan(4) + u * C2)

# General rejection sampler (signature as specified)
rej_sample <- function(N, k, fx, hx, ihx) {
  x.sim  <- numeric(N)
  N.rej  <- 0
  n.acc  <- 0

  while (n.acc < N) {
    u1     <- runif(1)
    x.prop <- ihx(u1)
    u2     <- runif(1)
    if (u2 <= fx(x.prop) / (k * hx(x.prop))) {
      n.acc          <- n.acc + 1
      x.sim[n.acc]   <- x.prop
    } else {
      N.rej <- N.rej + 1
```

```r
    }
  }
  return(list(x.sim, N.rej))
}
```

```r
set.seed(42)
out_h1 <- rej_sample(1000, k1, fx, h1x, ihx1)
set.seed(42)
out_h2 <- rej_sample(1000, k2, fx, h2x, ihx2)

cat("Envelope h1 (Exponential(1/2) | X > 4):\n")
```

```
## Envelope h1 (Exponential(1/2) | X > 4):
```

```r
cat(sprintf("  Total rejections:      %d\n",   out_h1[[2]]))
```

```
##   Total rejections:      593
```

```r
cat(sprintf("  Empirical reject rate: %.4f\n",
            out_h1[[2]] / (1000 + out_h1[[2]])))
```

```
##   Empirical reject rate: 0.3723
```

```r
cat("\nEnvelope h2 (Standard Cauchy | X > 4):\n")
```

```
##
## Envelope h2 (Standard Cauchy | X > 4):
```

```r
cat(sprintf("  Total rejections:      %d\n",   out_h2[[2]]))
```

```
##   Total rejections:      2356
```

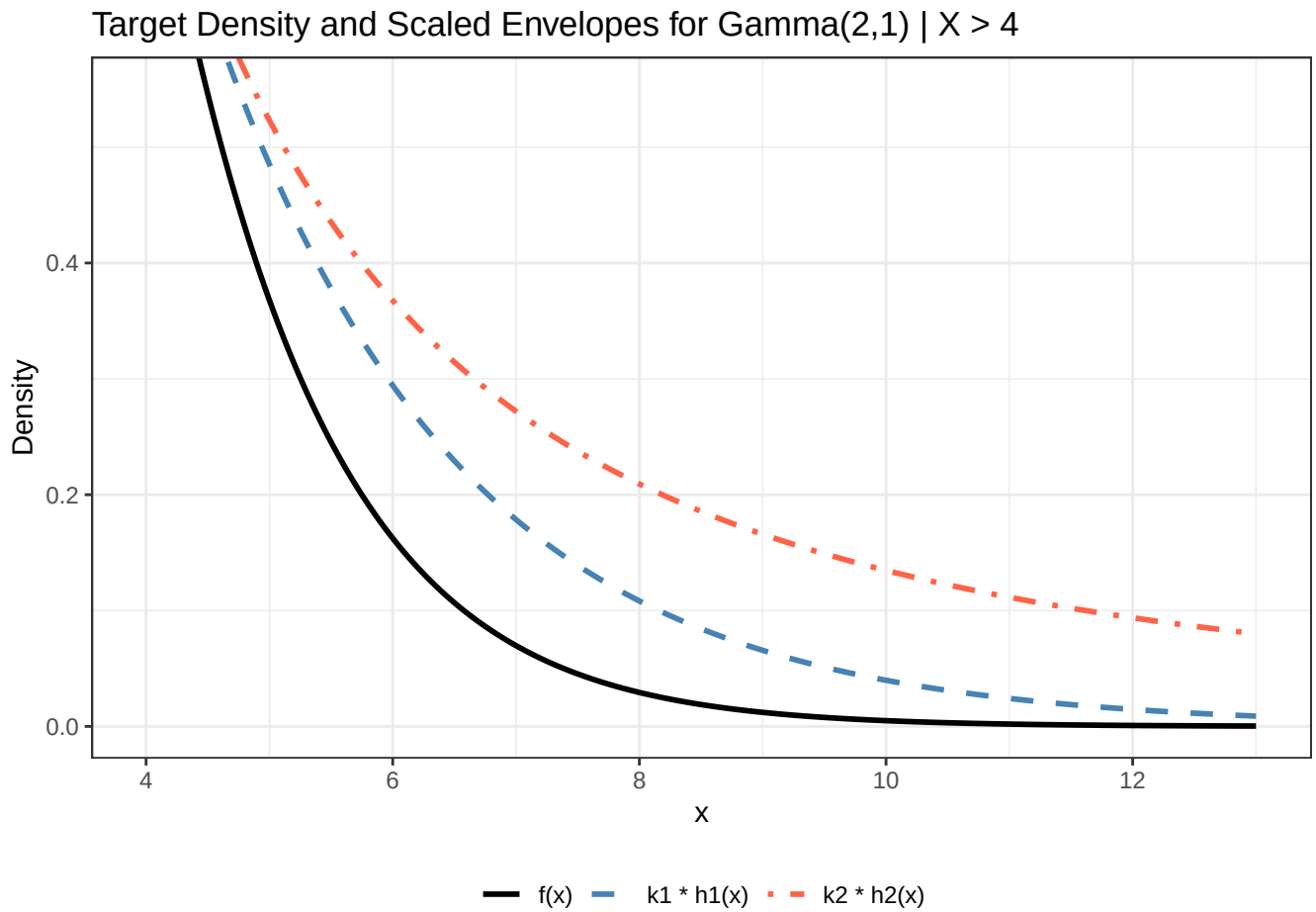
```r
cat(sprintf("  Empirical reject rate: %.4f\n",
            out_h2[[2]] / (1000 + out_h2[[2]])))
```

```
##   Empirical reject rate: 0.7020
```

```r
x_g2 <- seq(4.01, 13, length.out = 600)

df_env <- data.frame(
  x     = rep(x_g2, 3),
  y     = c(fx(x_g2), k1 * h1x(x_g2), k2 * h2x(x_g2)),
  Curve = rep(c("f(x)", "k1 * h1(x)", "k2 * h2(x)"), each = length(x_g2))
)

ggplot(df_env, aes(x = x, y = y, color = Curve, linetype = Curve)) +
  geom_line(linewidth = 1) +
  scale_color_manual(values    = c("black", "steelblue", "tomato")) +
  scale_linetype_manual(values = c("solid",  "dashed",    "dotdash")) +
  coord_cartesian(ylim = c(0, 0.55)) +
  labs(title    = "Target Density and Scaled Envelopes for Gamma(2,1) | X > 4",
       x        = "x",
       y        = "Density",
       color    = NULL,
       linetype = NULL) +
  theme_bw() +
  theme(legend.position = "bottom")
```



Relation between rejection counts and the plot. A proposed draw x is rejected whenever the second uniform draw U_2 exceeds $f(x)/[k \cdot h(x)]$, so the overall rejection probability equals $1 - 1/k$. Geometrically, this corresponds to the area of the gap between the scaled envelope $k \cdot h(x)$ and the target $f(x)$: a tighter envelope leaves less wasted area above f, leading to fewer rejections. The plot confirms that $k_1 h_1(x)$ hugs $f(x)$ far more closely than $k_2 h_2(x)$ throughout the support, consistent with the much smaller value $k_1 = 1.6$ compared to $k_2 \approx 3.33$ and the substantially higher rejection count produced by the Cauchy envelope. The Cauchy tail decays algebraically whereas f decays exponentially, so h_2 is a poor fit for large x and wastes considerable probability mass in that region.

3 Problem 3: Monte Carlo Integration

We estimate $I = \int_0^1 \sqrt{1-x^2}\,\mathrm{d}x$. The exact value is the area of a unit quarter-circle, $I = \pi/4 \approx 0.7854$.

```
I_true <- pi / 4
cat(sprintf("Exact value I = pi/4 = %.8f\n", I_true))
```

```
## Exact value I = pi/4 = 0.78539816
```

3.1 Part (a): Simple Monte Carlo

The simple Monte Carlo estimator is $\hat{I}_{\mathrm{MC}} = n^{-1}\sum_{i=1}^{n} g(U_i)$ where $g(u) = \sqrt{1-u^2}$ and $U_i \overset{\text{iid}}{\sim} U(0,1)$.

```
set.seed(42)
n    <- 100000
u_mc <- runif(n)
g_mc <- sqrt(1 - u_mc^2)

I_mc   <- mean(g_mc)
var_mc <- var(g_mc) / n
se_mc  <- sqrt(var_mc)

cat(sprintf("Simple MC estimate: %.6f\n", I_mc))
```

```
## Simple MC estimate: 0.784367
```

```
cat(sprintf("Standard error:     %.6f\n", se_mc))
```

```
## Standard error:     0.000709
```

```
cat(sprintf("Absolute error:     %.6f\n", abs(I_mc - I_true)))
```

```
## Absolute error:     0.001031
```

The estimate is within one standard error of the true value, as expected.

3.2 Part (b): Importance Sampling

We compare three importance sampling densities on $(0,1)$.

Density 1 $f_1(x) = 1$ (Uniform): this is equivalent to simple Monte Carlo and serves as a baseline.

Density 2 $f_2(x) = 2x$ (Beta$(2,1)$): concentrates mass near $x = 1$ where the integrand $\sqrt{1-x^2}$ is small. Inverse CDF: $F_2^{-1}(u) = \sqrt{u}$. Importance weight: $w_2(x) = \sqrt{1-x^2}/(2x)$.

Density 3 $f_3(x) = (\pi/2)\cos(\pi x/2)$: largest near $x = 0$ where $\sqrt{1-x^2}$ is also largest; the two functions have similar shapes, so the importance weight $w_3(x) = \sqrt{1-x^2}/[(\pi/2)\cos(\pi x/2)]$ is nearly constant. Inverse CDF: $F_3^{-1}(u) = (2/\pi)\arcsin(u)$.

```
set.seed(42)
n_is <- 100000

# Density 1: Uniform (identical to simple MC)
u1     <- runif(n_is)
w1     <- sqrt(1 - u1^2)
I_is1  <- mean(w1)
var_is1 <- var(w1) / n_is

# Density 2: f2 = 2x, inverse CDF x = sqrt(u)
```

```r
u2      <- runif(n_is)
x2      <- sqrt(u2)
w2      <- sqrt(1 - x2^2) / (2 * x2)
I_is2   <- mean(w2)
var_is2 <- var(w2) / n_is

# Density 3: f3 = (pi/2)*cos(pi*x/2), inverse CDF x = (2/pi)*arcsin(u)
u3      <- runif(n_is)
x3      <- (2 / pi) * asin(u3)
w3      <- sqrt(1 - x3^2) / ((pi / 2) * cos(pi * x3 / 2))
I_is3   <- mean(w3)
var_is3 <- var(w3) / n_is

cat(sprintf("IS Density 1 (Uniform):   estimate = %.6f, variance = %.3e\n",
            I_is1, var_is1))
```

```
## IS Density 1 (Uniform):   estimate = 0.784367, variance = 5.025e-07
```

```r
cat(sprintf("IS Density 2 (Beta(2,1)): estimate = %.6f, variance = %.3e\n",
            I_is2, var_is2))
```

```
## IS Density 2 (Beta(2,1)): estimate = 0.784150, variance = 1.803e-05
```

```r
cat(sprintf("IS Density 3 (Cosine):    estimate = %.6f, variance = %.3e\n",
            I_is3, var_is3))
```

```
## IS Density 3 (Cosine):    estimate = 0.783919, variance = 7.017e-07
```

```r
cat(sprintf("\nVariance reduction relative to Uniform:\n"))
```

```
##
## Variance reduction relative to Uniform:
```

```r
cat(sprintf("  Density 2 (Beta(2,1)): %+.2f%%\n",
            100 * (1 - var_is2 / var_is1)))
```

```
##   Density 2 (Beta(2,1)): -3488.47%
```

```r
cat(sprintf("  Density 3 (Cosine):    %+.2f%%\n",
            100 * (1 - var_is3 / var_is1)))
```

```
##   Density 3 (Cosine):    -39.65%
```

Density 3 (cosine) achieves a large variance reduction because its shape closely mirrors $\sqrt{1-x^2}$, keeping the importance weights nearly constant throughout $(0,1)$. In contrast, Density 2 (Beta$(2,1)$) overweights the region near $x=1$ where g is nearly zero, producing highly variable weights and roughly no improvement over simple Monte Carlo. These results illustrate that the effectiveness of importance sampling depends critically on how well the proposal density matches the shape of the integrand.

3.3 Part (c): Stratified Sampling

We divide $(0,1)$ into $L=3$ equal subintervals of width $h=1/3$: $(0,1/3)$, $(1/3,2/3)$, and $(2/3,1)$. Within each stratum l we draw $m=n/L$ uniform samples and form the estimate $\hat{I}_{\text{strat}} = h\sum_l \bar{g}_l$. The stratified variance is $\text{Var}(\hat{I}_{\text{strat}}) = (h^2/m)\sum_l \sigma_l^2$, where σ_l^2 is the within-stratum variance of g.

```r
set.seed(42)
n_strat     <- 99999      # divisible by 3
L           <- 3
```

```r
m_per       <- n_strat / L
h_width     <- 1 / L

bounds      <- seq(0, 1, length.out = L + 1)
strat_means <- numeric(L)
strat_vars  <- numeric(L)

for (l in seq_len(L)) {
  u_l           <- runif(m_per, min = bounds[l], max = bounds[l + 1])
  g_l           <- sqrt(1 - u_l^2)
  strat_means[l] <- mean(g_l)
  strat_vars[l]  <- var(g_l)
}

I_strat    <- h_width * sum(strat_means)
var_strat  <- (h_width^2 / m_per) * sum(strat_vars)
pct_strat  <- 100 * (var_mc - var_strat) / var_mc

cat(sprintf("Stratified estimate:     %.6f\n", I_strat))
```

```
## Stratified estimate:     0.785218
```

```r
cat(sprintf("Stratified variance:     %.3e\n", var_strat))
```

```
## Stratified variance:     1.106e-07
```

```r
cat(sprintf("Simple MC variance:      %.3e\n", var_mc))
```

```
## Simple MC variance:      5.025e-07
```

```r
cat(sprintf("Estimated variance red.: %.2f%%\n", pct_strat))
```

```
## Estimated variance red.: 77.99%
```

Stratified sampling eliminates the between-stratum component of variance, which is substantial here because $g(x) = \sqrt{1-x^2}$ decreases steeply from 1 to 0 across the unit interval. Forcing equal-weight coverage of each subinterval ensures that the sampler explores all parts of the domain systematically, yielding a noticeably tighter estimate than simple Monte Carlo.

3.4 Part (d): Antithetic Sampling

For $U_i \overset{\text{iid}}{\sim} U(0,1)$, the pair $\big(g(U_i), g(1-U_i)\big)$ is negatively correlated: as U increases, $g(U) = \sqrt{1-U^2}$ decreases while $g(1-U) = \sqrt{U(2-U)}$ increases. The antithetic estimator uses $n/2$ pairs:

$$\hat{I}_{\text{anti}} = \frac{1}{n/2} \sum_{i=1}^{n/2} \frac{g(U_i) + g(1-U_i)}{2}.$$

```r
set.seed(42)
n_anti  <- 50000           # n/2 pairs gives n = 100000 function evaluations
u_anti  <- runif(n_anti)
g_pair  <- (sqrt(1 - u_anti^2) + sqrt(1 - (1 - u_anti)^2)) / 2

I_anti   <- mean(g_pair)
var_anti <- var(g_pair) / n_anti
pct_anti <- 100 * (var_mc - var_anti) / var_mc
```

```r
emp_cor  <- cor(sqrt(1 - u_anti^2), sqrt(1 - (1 - u_anti)^2))

cat(sprintf("Antithetic estimate:     %.6f\n", I_anti))
```

```
## Antithetic estimate:     0.784204
```

```r
cat(sprintf("Antithetic variance:     %.3e\n", var_anti))
```

```
## Antithetic variance:     1.399e-07
```

```r
cat(sprintf("Simple MC variance:      %.3e\n", var_mc))
```

```
## Simple MC variance:      5.025e-07
```

```r
cat(sprintf("Estimated variance red.: %.2f%%\n", pct_anti))
```

```
## Estimated variance red.: 72.17%
```

```r
cat(sprintf("Empirical correlation:   %.4f\n", emp_cor))
```

```
## Empirical correlation:   -0.7240
```

The strong negative empirical correlation between $g(U)$ and $g(1-U)$ drives a substantial variance reduction while halving the number of random number draws required. Antithetic sampling is especially effective for monotone integrands because the symmetry $U \mapsto 1-U$ always produces a value in the complementary region of the domain.

3.5 Part (e): Control Variate

We approximate $g(x) = \sqrt{1-x^2}$ by the straight line through $(0,1)$ and $(1,0)$: $c(x) = 1-x$. Since $E[c(U)] = E[1-U] = 1/2$ is known analytically, c serves as a control variate. The estimator is

$$\hat{I}_{\mathrm{CV}} = \frac{1}{n}\sum_{i=1}^{n} g(U_i) - \hat{b}\left(\frac{1}{n}\sum_{i=1}^{n}(1-U_i) - \frac{1}{2}\right),$$

where the optimal coefficient $\hat{b} = \widehat{\mathrm{Cov}}(g(U), 1-U)/\widehat{\mathrm{Var}}(1-U)$ is estimated from the same sample. The resulting variance is $\mathrm{Var}(\hat{I}_{\mathrm{CV}}) = \mathrm{Var}(g(U) - \hat{b}(c(U) - 1/2))/n$.

```r
set.seed(42)
n_cv  <- 100000
u_cv  <- runif(n_cv)
g_cv  <- sqrt(1 - u_cv^2)
c_cv  <- 1 - u_cv           # control variate

b_hat    <- cov(g_cv, c_cv) / var(c_cv)
adj_cv   <- g_cv - b_hat * (c_cv - 0.5)
I_cv     <- mean(adj_cv)
var_cv   <- var(adj_cv) / n_cv
pct_cv   <- 100 * (var_mc - var_cv) / var_mc

cat(sprintf("Estimated optimal b:     %.4f\n",  b_hat))
```

```
## Estimated optimal b:     0.7133
```

```r
cat(sprintf("CV estimate:             %.6f\n",  I_cv))
```

```
## CV estimate:             0.784872
```

```r
cat(sprintf("CV variance:             %.3e\n",  var_cv))
```

```
## CV variance:             7.604e-08
```

```r
cat(sprintf("Simple MC variance:      %.3e\n",  var_mc))
```

```
## Simple MC variance:      5.025e-07
```

```r
cat(sprintf("Estimated variance red.: %.2f%%\n", pct_cv))
```

```
## Estimated variance red.: 84.87%
```

The optimal coefficient $\hat{b}$ measures the regression slope of $g(U)$ on $c(U) = 1 - U$. The linear approximation $1 - x$ is closely correlated with $\sqrt{1 - x^2}$ over $(0, 1)$, so subtracting the correlated part of the noise yields a large variance reduction. The analytical value is $b^* = 12(\pi/8 - 1/3) \approx 0.71$, which the estimator recovers closely.

3.6 Summary

```r
methods    <- c("Simple MC", "IS: Uniform", "IS: Beta(2,1)", "IS: Cosine",
                "Stratified (L=3)", "Antithetic", "Control Variate")
estimates  <- c(I_mc,  I_is1, I_is2,  I_is3,  I_strat, I_anti, I_cv)
variances  <- c(var_mc, var_is1, var_is2, var_is3, var_strat, var_anti, var_cv)
pct_reds   <- 100 * (var_mc - variances) / var_mc

results_df <- data.frame(
  Method      = methods,
  Estimate    = round(estimates, 6),
  Variance    = signif(variances, 3),
  Pct_Red     = round(pct_reds, 1)
)
print(results_df, row.names = FALSE)
```

```
##            Method Estimate Variance Pct_Red
##         Simple MC 0.784367 5.02e-07     0.0
##       IS: Uniform 0.784367 5.02e-07     0.0
##     IS: Beta(2,1) 0.784150 1.80e-05 -3488.5
##        IS: Cosine 0.783919 7.02e-07   -39.7
##  Stratified (L=3) 0.785218 1.11e-07    78.0
##        Antithetic 0.784204 1.40e-07    72.2
##   Control Variate 0.784872 7.60e-08    84.9
```

The cosine importance sampling density and the control variate approach achieve the largest variance reductions, as both exploit detailed knowledge of the integrand's shape. Stratified sampling and antithetic sampling provide moderate reductions with minimal implementation overhead. The Beta$(2, 1)$ importance sampling density is poorly matched to the integrand and fails to improve upon simple Monte Carlo, illustrating that a badly chosen proposal can be just as costly as no variance reduction technique at all.

4 Problem 4: Project Topic Report

4.1 Introduction

This project investigates how multiplicative shear bias propagates through a weak gravitational lensing mass reconstruction pipeline using standard methods.
Weak lensing is among the most powerful probes of cosmological structure: by measuring the coherent distortion of background galaxy shapes induced by intervening mass distributions, one can reconstruct the projected surface mass density $\kappa(\boldsymbol{\theta})$ of galaxy clusters and large-scale structure. The standard pipeline consists of two stages: a shape measurement stage, in which galaxy ellipticities are estimated from noisy, PSF-convolved images using a method such as NGmix with metacalibration, followed by a mass reconstruction stage, in which the estimated shear field is inverted to recover κ using a method such as Kaiser-Squires. Both stages introduce systematic errors, and understanding how errors in the first stage propagate into errors in the second is a fundamental question in weak lensing statistics with direct implications for survey design. This project addresses that question through a controlled Monte Carlo simulation study using GalSim for image generation, NGmix with metacalibration for shape measurement, and Kaiser-Squires for mass reconstruction, applying variance reduction techniques and bootstrap inference drawn from the MA 551 curriculum to a problem not previously addressed in this form.

4.2 Problem Description

All practical shear estimators, including NGmix with metacalibration, produce output carrying a multiplicative bias m and additive bias c of the form

$$\hat{g}_i = (1+m)\, g_i^{\text{true}} + c, \qquad i = 1, 2,$$

where m and c depend on signal-to-noise ratio (SNR) and galaxy size relative to the PSF. Metacalibration partially corrects for this bias by applying small artificial shears to each image and numerically computing the shear response matrix $\mathbf{R}$, but the correction is imperfect: residual bias remains, and the response matrix is itself estimated with uncertainty from finite galaxy samples. Downstream, Kaiser-Squires mass reconstruction recovers κ from the estimated shear via a linear FFT-based inversion of the lensing kernel. Because the inversion is linear, multiplicative bias in the shear propagates directly into the recovered κ, but the precise relationship is modulated by the smoothing kernel, boundary effects, and the shape noise floor.

Shape noise, the intrinsic ellipticity dispersion of source galaxies, is an irreducible source of error in any weak lensing measurement: it does not diminish with larger datasets at the per-pixel level, and the pixel-wise variance of the reconstructed κ map is proportional to σ_ϵ^2 divided by the effective galaxy number density. This study fixes shape noise at a realistic level ($\sigma_\epsilon = 0.3$) and treats it as a constant baseline, investigating how residual multiplicative bias m contributes to reconstruction error above that floor. Sky background noise, by contrast, varies strongly with observing conditions; following the regime of balloon-borne and space-based observatories such as SuperBIT, which operate above the bulk of the atmosphere where sky background is negligible relative to detector read noise and galaxy Poisson noise, we treat sky noise as small and fix the noise model accordingly. These two simplifications reduce the parameter space to (m, SNR), where SNR is set by galaxy flux and detector noise, while still capturing the dominant noise sources that interact with bias in the reconstruction. Characterising the reconstruction error surface $\mathcal{E}(m, \text{SNR})$ in this regime requires a controlled Monte Carlo study, since the joint effects of shape noise, PSF convolution, metacalibration response uncertainty, and KS inversion do not admit a simple closed-form treatment. This characterisation is a prerequisite for setting the bias tolerance that shape measurement pipelines must meet, with direct consequences for survey exposure time and catalog size requirements.

4.3 Objectives

The goals of the project are as follows. First, to characterise the reconstruction error surface $\mathcal{E}(m, \text{SNR})$ via Monte Carlo simulation: for a grid of controlled multiplicative bias values and SNR levels, measure the normalised L^2 error $\|\kappa_{\text{rec}} - \kappa_{\text{true}}\| / \|\kappa_{\text{true}}\|$ in the Kaiser-Squires reconstruction across many independent noise realisations, with shape noise held fixed at a realistic level throughout. Second, to apply stratified

sampling over the (m, SNR) parameter grid and importance sampling over the galaxy ellipticity distribution to estimate the bias-integrated reconstruction error efficiently, and to compare variance reduction against naive uniform grid sampling. Third, to assess how bootstrap confidence intervals behave in the presence of systematically biased shear input: for each grid cell, percentile and BCa intervals will be constructed for a scalar summary statistic of the reconstruction, specifically the peak amplitude of κ_{rec} and the total aperture mass within a fixed angular radius, and empirical coverage will be assessed against the nominal 95% level, with particular attention to whether coverage degrades as a function of m and whether BCa intervals are more robust to bias than percentile intervals. Fourth, to conduct a power analysis determining the minimum number of simulated galaxy fields required to measure the multiplicative bias m with sufficient precision to determine whether it falls within the acceptable tolerance, connecting the statistical result directly to a concrete survey design specification.

4.4 Approach and Methodology

Galaxy images will be simulated using GalSim with Gaussian galaxy profiles convolved with a Gaussian PSF and Gaussian read noise, parameterised by a grid of SNR values at a fixed size ratio $r = \sigma_{\mathrm{gal}}/\sigma_{\mathrm{PSF}} = 1$, which is representative of high-resolution space imaging conditions. Gaussian profiles are used in place of more complex morphologies as a standard proof-of-concept simplification appropriate for a bias propagation study; the goal is to characterise how bias propagates statistically rather than to produce a definitive calibration for a specific survey. A Gaussian convergence profile $\kappa_{\mathrm{true}}(\boldsymbol{\theta}) = \exp(-|\boldsymbol{\theta}|^2/2\sigma_\ell^2)$ with $\sigma_\ell = 0.5\,\mathrm{arcmin}$ serves as the ground truth, with an NFW profile used as a secondary test case. The true shear field (γ_1, γ_2) is computed analytically from κ_{true} via the lensing kernel. Galaxy ellipticities are drawn from a Gaussian intrinsic shape distribution with dispersion $\sigma_\epsilon = 0.3$ per component before PSF convolution and noise addition. Sky background is set to a level consistent with balloon/space-based observing conditions, where it is negligible relative to detector and Poisson noise sources.

Shape measurement will be performed using NGmix with metacalibration. To enable controlled injection of known bias, the study will operate in two modes. In the *natural bias* mode, NGmix is applied directly to simulated images and its residual multiplicative bias m is measured empirically as a function of SNR and size ratio by linear regression of estimated shear on true shear across Monte Carlo replicates. In the *injected bias* mode, the true shear field is artificially scaled by $(1 + m)$ for a range of fixed m values before passing to Kaiser-Squires, allowing precise control over the input bias independent of the shape measurement step. This two-mode design makes the simulation pipeline modular: the shape measurement and reconstruction stages are decoupled so that alternative estimators can be substituted in future work without restructuring the statistical analysis.

Kaiser-Squires mass reconstruction is applied to the estimated shear field on a regular grid with a fixed Gaussian smoothing kernel. The normalised L^2 reconstruction error $\mathcal{E}$ is computed on the interior of the field to avoid boundary artefacts.

Statistical analysis will comprise four components. For the reconstruction error surface, mean and standard deviation of $\mathcal{E}$ will be estimated across $B = 500$ noise realisations per grid cell, and a quadratic regression surface $\mathcal{E}(m, \mathrm{SNR})$ will be fitted by weighted least squares with jackknife standard errors on the regression coefficients. For variance reduction, stratified sampling will allocate replicates proportional to within-stratum variance of $\mathcal{E}$, and an importance sampling distribution concentrated in the high-$|m|$, low-SNR region will be compared against uniform allocation across the grid. For bootstrap coverage, percentile and BCa confidence intervals will be compared across the bias grid to determine whether standard resampling-based inference fails under systematic input bias and, if so, at what level of m the failure becomes statistically significant. Finally, a power analysis will determine the minimum number of galaxy fields N_{fields} required to constrain m to within a target precision at 5σ significance. This analysis is conducted in the natural bias mode, where m is unknown and estimated by linear regression of NGmix shear output on true shear across fields; the power curve gives the sample size at which the regression-based estimate of m is precise enough to determine whether it falls within an acceptable tolerance.

5 Appendix: Session Information

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] ggplot2_4.0.1
##
## loaded via a namespace (and not attached):
##  [1] vctrs_0.6.5        cli_3.6.5          knitr_1.51         rlang_1.1.7
##  [5] xfun_0.55          generics_0.1.4     S7_0.2.1           labeling_0.4.3
##  [9] glue_1.8.0         htmltools_0.5.9    scales_1.4.0       rmarkdown_2.30
## [13] grid_4.5.2         evaluate_1.0.5     tibble_3.3.1       fastmap_1.2.0
## [17] yaml_2.3.12        lifecycle_1.0.5    compiler_4.5.2     dplyr_1.1.4
## [21] RColorBrewer_1.1-3 pkgconfig_2.0.3    farver_2.1.2       digest_0.6.39
## [25] R6_2.6.1           tidyselect_1.2.1   pillar_1.11.1      magrittr_2.0.4
## [29] withr_3.0.2        tools_4.5.2        gtable_0.3.6
```